

ON IDENTIFIABILITY IN TRANSFORMER NEURAL NETWORKS

FINLO BANARSE, ARDEN DIAKHATE-PALME, LOGAN HINKLEY, FINLAY SCHEIB

(SUPERVISOR: ANDREAS GRIVAS)

Abstract. Transformer neural networks are the foundation of modern day LLMs. By processing massive amounts of information, they learn to generate a coherent response to inputs. The network pulls information together by learning representations and contexts between words, producing attention distributions. These distributions are not always identifiable, meaning multiple different attention distributions can produce the same output. This is caused by the left null space of the learnt weights being non empty. We visualise this issue on the probability simplex and provide a set of alternative weights that produce the same output for given examples. To address this, [4] proposes effective attention, which we compare with raw attention using Pearson’s correlation coefficient, where differences arise when the left null space is non-empty. Another problem that occurs is the low-rank bottleneck in the attention heads. This describes distributions that cannot be produced by outputs of softmax. This happens when the head sizes are less than the input token sequence length. We introduce the low-rank bottleneck formally and provide a novel derivation of a condition that can be used to verify if a distribution can be reached. A solution to avoid the low-rank bottleneck is proposed in [3] called “Fixed Multi-Head Attention”. In our research, this is implemented into NanoGPT, where the model sees improved performance. We then explore how the low-rank bottleneck can be removed for a single column of attention using identifiability, but it still remains a key limitation in the transformer architecture when considering the full matrix.

1. Introduction. Transformers are the neural network architecture that has formed the foundation for the AI and large language model (LLM) revolution. They first appeared in the literature in 2017 in a famous paper titled “Attention is all you need” [13]. Whilst they contain many different types of layers, including feed-forward and encoder layers, their strength lies in the *Multi-Head Attention* layers. These are said to be *context enriching* as they allow the model to understand the relationship between words and their context, rather than view sentences as a bag of words. These attention heads are the object of interest for this project, during which we investigated two key problems: namely, identifiability and expressiveness.

Identifiability deals with the question: under what conditions are the attention weights unique? This is interesting from an interpretability standpoint, as given an output it is desirable to know exactly the unique attention weights that led to it. Secondly, expressiveness is a question of existence; given an embedding of our tokens is every possible attention output achievable? We will see that the attention weight vectors are themselves probability distributions and so the question becomes: given an input sequence what probability distributions are reachable? Finally, we investigate the relationship between the two problems and investigate if the non-uniqueness of the attention weights can help solve the non-existence of some distributions. For our experiments, we used NanoGPT, a transformer-based model [13], using a set of character-level token embeddings from Shakespeare’s Complete Works as our dataset [10]. Although this model is a simplification of modern LLMs, it remains useful in our analysis.

The report is outlined as follows: Section 2 provides some background with a crash course in attention heads and embedding types. Section 3 addresses attention identifiability, provides some examples, and proposes a solution, namely effective attention. Section 4 explores the low-rank bottleneck, when it exists, and how it can be avoided. Finally, Section 5 concludes the report with a discussion on the experiments conducted in NanoGPT and an attempt to use the non-identifiability to fix the non-existence of some distributions.

2. Attention: How does it work?.

2.1. Types of Embedding. For transformer models, the dataset is usually raw text, and before it can be used it must be broken up into tokens. Several methods can be used to tokenise text such as GPT2 BPE [12] and WordPiece [6], but we consider each character as a token since we lack access to GPU clusters. Next, we represent each token as a vector in $\mathbb{R}^d$, as transformer models cannot handle strings. We want this embedding vector to capture both some meaning of the token and also its position in a ‘lookback window’ of n previous tokens. So, transformer-based models must inject some information about the relative or absolute position of the tokens in a sequence.

Consider a dataset of English words from a dictionary, D , with a vocabulary size of $V := |D|$. The token embedding layer learns an affine function $\mathcal{F}_{\text{tok}} : \mathbb{R}^V \rightarrow \mathbb{R}^d$, while the position embedding layer learns an affine function $\mathcal{F}_{\text{pos}} : \mathbb{R}^n \rightarrow \mathbb{R}^d$. The sum of the two token embeddings is then our embedding matrix $\mathbf{E} \in \mathbb{R}^{n \times d}$, which is fed into the series of transformer blocks. Note that by ‘learn’ we refer to the training process of minimising the global model loss function (see Figure 5.1) to determine optimal weights $\mathbf{W}$ and the $\mathbf{b}$ through back-propagation in $\mathcal{F}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}$ for both embedding layers. On the other hand, cosine-similarity could also be used to determine positional encodings [13], however, we focus on the previously discussed affine function method since the GPT model in our experiments uses this.

More specifically, in the NanoGPT implementation used for our experiments, the first two layers of the model are `torch.nn.Embedding()`, which store embeddings of a fixed dictionary and size. These functions serve as lookup tables, allowing the right tokens to be retrieved in generation. These layers implement a one-to-one function $\mathcal{F}_{\text{tok}} : \mathbb{Z} \rightarrow \mathbb{R}^d$ and a function $\mathcal{F}_{\text{pos}} : \{1, \dots, n\} \rightarrow \mathbb{R}^d$ where v is the vocabulary size, and n is the sequence length. In order to train the model on a multicore laptop CPU in a reasonable amount of time, we encode each character in Shakespeare via $\mathcal{F}_{\text{pre}} : C \rightarrow \mathbb{Z}$ where

$$(2.1) \quad C = [! \$ \% ' , - . 3 : ; ? \text{ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz}],$$

is a list of unique characters that appear in the text. The vocabulary size is the length of C , which in this case is 65. Note that $\mathcal{F}_{\text{pre}}$ is one-to-one because each character is mapped to an integer via its unique ASCII code [2].

Consider a set B of b sections (batches) from the dataset, each consisting of n contiguous characters. So $B' = \mathcal{F}_{\text{pre}}(B) \in \mathbb{R}^{n \cdot b}$ corresponds to the integer encodings of those $|B| = n \cdot b$ characters. The input to

our series of transformer layers in Figure 5.1 is then $\mathbf{E} = \mathcal{F}_{\text{tok}}(B') + \mathcal{F}_{\text{pos}}\left(\begin{bmatrix} 1, \dots, n \end{bmatrix}^\top\right) \in \mathbb{R}^{d \times (n \cdot b)}$. Note that in our analysis, we only consider one batch ($b = 1$), hence $\mathbf{E} \in \mathbb{R}^{d \times n}$.

2.2. Single Head attention. Before diving into how we construct a single head of attention we begin by defining two very important notions, namely, the softmax function and the probability simplex. Firstly, we define softmax,

DEFINITION 2.1 (Softmax). *Given a vector $\mathbf{x} \in \mathbb{R}^m$, then*

$$\text{softmax}(\mathbf{x})_i = \frac{\exp(x_i)}{\sum_{j=1}^m \exp(x_j)}.$$

Firstly, the definition of the softmax function is only defined for vectors, thus when applying it to a matrix, we must apply it row-wise or column-wise. It is also worth noting that the output of the softmax function is a valid probability distribution. That is, the role of the softmax function is to turn very negative entries of a vector into very low probability ones and very large entries into high probability ones. If $\mathbf{y} = \text{softmax}(\mathbf{x})$ for $\mathbf{x} \in \mathbb{R}^m$, then $\mathbf{y} \in \mathbb{R}^m$ but with the added properties that $\mathbf{y}^\top \mathbf{1} = 1$ and $\mathbf{y} \geq \mathbf{0}$. Thus, $\mathbf{y}$ is a valid probability distribution and lives in the $m - 1$ dimensional probability simplex.

DEFINITION 2.2 (Probability Simplex). *The m -dimensional probability simplex is given by:*

$$\Delta^m = \left\{ \mathbf{x} \in \mathbb{R}^{m+1} \mid \sum_{i=1}^{m+1} x_i = 1, \mathbf{x} \geq \mathbf{0} \right\}.$$

Intuitively, all vectors that are positive and sum to one form a probability simplex; which is a hyper-surface always of dimension one less than the dimension of the ambient vector space. Some examples of the simplex in $\mathbb{R}^2, \mathbb{R}^3$ and $\mathbb{R}^4$ are shown in Figure 2.1. These two notions will be important for the construction of single head attention.

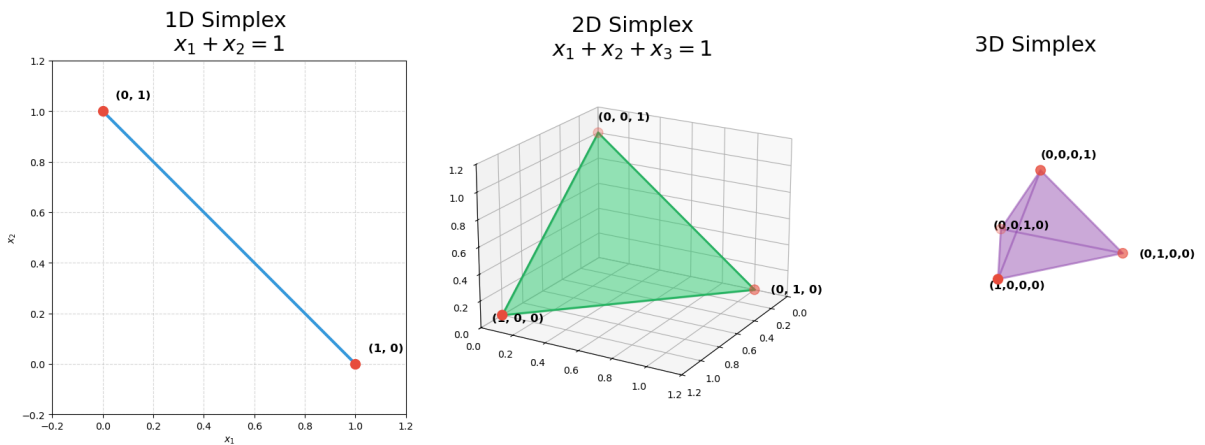


FIGURE 2.1. Some examples of the probability simplex in $\mathbb{R}^2, \mathbb{R}^3$ and $\mathbb{R}^4$.

89

90 Once we have the embedding matrix $\mathbf{E} \in \mathbb{R}^{d \times n}$, where n denotes the sequence length, i.e. the number of
 91 tokens we have in the sentence, this is then used to calculate the three key matrices for attention, namely
 92 the key, query and value matrices $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{d_q \times n}, \mathbf{V} \in \mathbb{R}^{n \times d_v}$. These are given by

$$93 \quad \mathbf{K} = \mathbf{W}_\mathbf{K} \mathbf{E}, \quad \mathbf{Q} = \mathbf{W}_\mathbf{Q} \mathbf{E}, \quad \mathbf{V} = \mathbf{W}_\mathbf{V} \mathbf{E},$$

94 where the corresponding weight matrices, $\mathbf{W}_\mathbf{K}, \mathbf{W}_\mathbf{Q}, \mathbf{W}_\mathbf{V}$, are learnt by the model during training in order
 95 to produce the most optimal key, query and value matrices. Once the three matrices have been calculated,
 96 the output of a single head of attention is given by,

$$97 \quad (2.2) \quad \text{Attention}(\mathbf{K}, \mathbf{Q}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{K}^\top \mathbf{Q}}{\sqrt{d_q}} \right) \mathbf{V}.$$

98 As discussed before, the output of the softmax function is a probability distribution, and so the output of
 99 attention is a weighted sum of value vectors with the corresponding weights of each token stored in the
 100 attention matrix,

$$101 \quad \mathbf{A} = \text{softmax} \left(\frac{\mathbf{K}^\top \mathbf{Q}}{\sqrt{d_q}} \right).$$

102 These attention weights are calculated by applying softmax to the matrix $\mathbf{K}^\top \mathbf{Q} \in \mathbb{R}^{n \times n}$, which can be
 103 interpreted as taking dot products between columns of the $\mathbf{Q}$ and $\mathbf{K}$ matrices, respectively. Thus, if
 104 this dot product is large, it will lead to a large probability in the attention weights $\mathbf{A}$, and hence more
 105 weighting of the corresponding value vector in the output of the attention head. This interpretation helps
 106 give some intuition for what exactly the key and query matrices are. They are the critical objects that
 107 help the model understand the context of the sentence and which tokens ‘attend’ to others, i.e. which
 108 words in the sentence have strong importance to one another. Some like to interpret the key and query
 109 matrices as the query vector for a given token asking a question of all other tokens, say “*Is there an*
 110 *adjective before my token?*”. The dot product can then be viewed as key vectors of every other token
 111 answering this query. If the answer is “*Yes! I am an adjective before this word.*”, then the corresponding
 112 dot product will be large and hence weight the value vector more heavily. Whilst this is only a loose
 113 analogy, it helps to understand how the key and query work together to identify the most relevant tokens
 114 and enrich a sequence of tokens with meaning.

115 What has been described above is only a single head of attention, however, most models and particularly
 116 LLMs, run multiple heads of attention in parallel to further enrich the context. This is known as *multi-head*
 117 *attention*.

118 **2.3. Multi-Head Attention.** The standard for transformer neural networks is to use multi-head
 119 attention. In this setup, typically, h heads are chosen, with head or *embedding size* to be $d_v = \frac{d}{h}$. Note

that the overall embedding dimension d is equal to the sum of the individual head sizes. The tokens are then projected down to d_v dimensions in each head, and attention is calculated in parallel. The h attention outputs are then concatenated and multiplied by some matrix $\mathbf{H}_M$ to combine the attention outputs. This is then added to the original embedding, $\mathbf{E}$, and normalised. The multi-head setup can be described by the following,

$$\text{head}(\mathbf{E})_i = \text{softmax}\left(\frac{\mathbf{K}_i^T \mathbf{Q}_i}{\sqrt{d_v}}\right) \cdot \mathbf{V}_i,$$

$$\text{MultiHead}(\mathbf{E}) = \text{Concat}[\text{head}(\mathbf{E})_1, \dots, \text{head}(\mathbf{E})_h],$$

where $\text{head}(\mathbf{E})_i$ denotes the attention matrix, $\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$, produced by a head. This gives us a layer output of

$$\mathbf{Z} = \text{LayerNorm}(\mathbf{E} + \text{MultiHead}(\mathbf{E}) \cdot \mathbf{H}_M).$$

Projecting onto multiple heads allows the model to capture more nuance than a single attention head by combining different representations learnt by individual heads, noted by [13]. Furthermore, as attention heads are run in parallel, this is highly computationally efficient and allows one to scale the model up significantly.

3. Attention Identifiability. As discussed above, the output of multi-head attention is the output of h single heads of attention run in parallel, which can be framed as

$$(3.1) \quad \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})\mathbf{H} = \mathbf{AVH} = \mathbf{AT},$$

where $\mathbf{T} = \mathbf{VH} \in \mathbb{R}^{n \times d}$ and $\mathbf{H} \in \mathbb{R}^{d_v \times d}$. Where $\mathbf{H}$ is the matrix that maps each single head of attention back to the full space; it can thus be viewed as the output projection. Similarly, $\mathbf{T}$ can be viewed as the value matrix $\mathbf{V}$, projected into the full model space. Hence, the output of multi-head attention is a weighted sum of the $\mathbf{T}$ vectors, where the weights are found in $\mathbf{A}$, summed over each head.

To explore the uniqueness of $\mathbf{A}$, we must investigate the left null space of $\mathbf{T}$, denoted $\text{LN}(\mathbf{T})$, since our attention vectors multiply the matrix $\mathbf{T}$ on the left.

DEFINITION 3.1. *The left null space of $\mathbf{T}$ is given by,*

$$\text{LN}(\mathbf{T}) = \{x \in \mathbb{R}^{1 \times n} : x\mathbf{T} = 0\}.$$

If we construct a matrix with rows $\mathbf{x}_i \in \text{LN}(\mathbf{T})$, $\tilde{\mathbf{A}} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^T$, then we have that $\tilde{\mathbf{A}}\mathbf{T} = 0$ and so $(\mathbf{A} + \tilde{\mathbf{A}})\mathbf{T} = \mathbf{AT}$. That is, we can find a matrix $\tilde{\mathbf{A}}$ to add to our original attention matrix $\mathbf{A}$ that achieves the same output. Importantly, $\tilde{\mathbf{A}}$ must be constructed with vectors in $\text{LN}(\mathbf{T})$. This demonstrates that the non-identifiability of the attention weights is equivalent to the left null space of $\mathbf{T}$ being non-empty. Below, we state the condition for the attention weights being non-identifiable.

THEOREM 3.2. The attention weight matrix $\mathbf{A}$ is non-identifiable when the input sequence length, n , is larger than the attention head dimension, d_v . Equivalently,

$$\dim(\text{LN}(\mathbf{T})) = \begin{cases} n - d_v & \text{if } n > d_v \\ 0 & \text{otherwise} . \end{cases}$$

Proof. We can calculate an upper bound on the rank of $\mathbf{T}$,

$$\text{rank}(\mathbf{T}) \leq \min(\text{rank}(\mathbf{V}), \text{rank}(\mathbf{H})) = \min(n, d_v, d_v, d) = \min(n, d_v).$$

Then, using the rank nullity theorem for the left null space, we get

$$\dim(\text{LN}(\mathbf{T})) = n - \text{rank}(\mathbf{T}) \geq n - \min(n, d_v) = \begin{cases} n - d_v & \text{if } n > d_v \\ 0 & \text{otherwise} , \end{cases}$$

where this is an equality statement if the matrices $\mathbf{V}$ and $\mathbf{H}$ are of full rank. Here, we have used an adjusted version of the rank-nullity theorem for the left null space, which satisfies $\text{LN}(\mathbf{T}) = \text{Null}(\mathbf{T}^T)$. Thus, it becomes the number of rows (not columns) minus the rank of $\mathbf{T}$. $\square$

However, we have not yet exploited the fact that the attention weights are themselves probability distributions, that is $\mathbf{A}\mathbf{1} = \mathbf{1}$ and $\mathbf{A} \geq \mathbf{0}$. We of course want any alternative set of attention weights, $\mathbf{A} + \tilde{\mathbf{A}}$, to also be constrained to the probability simplex, that is $(\mathbf{A} + \tilde{\mathbf{A}})\mathbf{1} = \mathbf{1}$ and $\mathbf{A} + \tilde{\mathbf{A}} \geq \mathbf{0}$. This leads to a set of three conditions on $\tilde{\mathbf{A}}$ in order to construct an alternative attention weight,

$$(3.2) \quad a) \quad \tilde{\mathbf{A}}\mathbf{T} = \mathbf{0}, \quad b) \quad \tilde{\mathbf{A}}\mathbf{1} = \mathbf{0}, \quad c) \quad \tilde{\mathbf{A}} \geq -\mathbf{A}.$$

The first two of these can be combined into $\tilde{\mathbf{A}}$ living in the left null space of the augmented matrix $[\mathbf{T}, \mathbf{1}]$. Theorem 3.2 gave us a condition on the dimension of $\text{LN}(\mathbf{T})$, but we now have an extra column which at most increases the rank by one. Thus, the upper bound is now $\text{rank}([\mathbf{T}, \mathbf{1}]) \leq \min(n, d_v) + 1$. However, another fundamental rule is that the rank of a matrix can be no more than the number of rows or columns. Since $[\mathbf{T}, \mathbf{1}] \in \mathbb{R}^{n \times (d+1)}$ and $d_v \leq d$, it has n rows and $d + 1$ columns. Thus, we can have $\text{rank}([\mathbf{T}, \mathbf{1}]) = d_v + 1$, but not $\text{rank}([\mathbf{T}, \mathbf{1}]) = n + 1$, hence we can re-write the upper bound as $\text{rank}([\mathbf{T}, \mathbf{1}]) \leq \min(n, d_v + 1)$. Thus, applying the rank-nullity theorem now to $[\mathbf{T}, \mathbf{1}]$, we achieve

$$(3.3) \quad \dim(\text{LN}([\mathbf{T}, \mathbf{1}])) = n - \text{rank}([\mathbf{T}, \mathbf{1}]) \geq n - \min(n, d_v + 1) = \begin{cases} n - d_v - 1 & \text{if } n - d_v > 1 \\ 0 & \text{otherwise} . \end{cases}$$

Thus, the condition for non-identifiability has now changed when we constrain the attention weights to the simplex. We now need the sequence length minus the attention head dimension to be greater than 1.

3.1. Methods for Finding Alternative Weights. Above we have shown that if we have $n - d_v > 1$, then the first two conditions on $\tilde{\mathbf{A}}$ in Equation (3.2) are satisfied, but what about the last one? Informally,

the idea is to find a ball with radius λ , where λ is chosen such that all the alternative weights lie within this ball around the origin. Then the intersection of this ball and the null space provides the full set of alternative weights. The work in [4] formalises this thinking. Let $\mathbf{a}$ be a row of $\mathbf{A}$ and $\tilde{\mathbf{a}}$ of $\tilde{\mathbf{A}}$, so of course $\tilde{\mathbf{a}} \in \text{LN}([\mathbf{T}, \mathbf{1}])$. Then, since $\mathbf{a} \geq \mathbf{0}$, all positive elements of $\tilde{\mathbf{a}}$ automatically satisfy $\tilde{\mathbf{a}} \geq -\mathbf{a}$. Thus, consider only the negative components $\tilde{a}_i < 0$, we need to find some constant $\lambda_i > 0$ that shrinks it to satisfy the condition, $\lambda_i \tilde{a}_i \geq -a_i$. Thus we find the λ for which this is satisfied for all i with $\tilde{a}_i < 0$. This is given by,

$$\lambda_{max} = \min_{i \in I} \left(\frac{-a_i}{\tilde{a}_i} \right), \quad I = \{i | \tilde{a}_i < 0\}.$$

Thus, any vector in the set $\{\lambda \tilde{\mathbf{a}} | 0 \leq \lambda \leq \lambda_{max}\}$ satisfies the third and final condition, hence, it produces a valid alternative weight when added to $\mathbf{a}$. That is, for $\mathbf{a}' \in \{\lambda \tilde{\mathbf{a}} | 0 \leq \lambda \leq \lambda_{max}\}$, then $(\mathbf{a}' + \mathbf{a})\mathbf{T} = \mathbf{a}\mathbf{T}$. We show how this works in practice below.

3.2. Examples. Using the approach from [4] as explained above, we calculate the set of alternative weights for a given matrix and distribution. Suppose our $\mathbf{T}$ matrix is given by,

$$\mathbf{T} = \begin{bmatrix} 1.4 & 0.2 \\ 0.5 & 1.5 \\ -1.2 & 0.9 \\ 0.1 & 0.8 \end{bmatrix} \in \mathbb{R}^{4 \times 2}.$$

We can already see there is going to be non-identifiability since in this case we have a sequence length of $n = 4$ and $d_v = 2$, thus $n - d_v > 1$. Suppose we are given the attention vector, $\mathbf{a} = [0.25, 0.25, 0.25, 0.25]^T$, which is the uniform distribution in $\mathbb{R}^4$ at the centre of the 3-simplex. Here $\mathbf{a}^T \mathbf{T} = [0.2, 0.85]^T$ is the output and we want to find the alternative distributions that give the same output. Firstly, $\mathbf{T}$ is augmented with the vector of ones, before solving to find some $\tilde{\mathbf{a}} \in \text{LN}([\mathbf{T}, \mathbf{1}])$. This was done in SciPy which provides the basis vectors for the null space. In this example, $\text{rank}(\mathbf{T}) = 2$, so $\text{rank}([\mathbf{T}, \mathbf{1}]) = 3$, thus we expect the left null space to be of dimension $4 - 3 = 1$ by the rank-nullity theorem. We identify the entry as $\tilde{\mathbf{a}} = [-0.297, -0.203, -0.360, 0.861]^T$. This vector satisfies the first two conditions in Equation (3.2), meaning that $\mathbf{a}^T \mathbf{T} = (\mathbf{a} + \tilde{\mathbf{a}})^T \mathbf{T}$. We now need to constrain $\tilde{\mathbf{a}}$ to the probability simplex, making sure that it satisfies $\tilde{\mathbf{a}} \geq -\mathbf{a}$. To enforce this condition we compute the scaling factor λ_{max} using Equation (3.1), which leads to $\lambda_{max} = 0.695$. Additionally, if $\tilde{\mathbf{a}} \in \text{LN}([\mathbf{T}, \mathbf{1}])$ then so is $-\tilde{\mathbf{a}}$. Thus, the process is repeated finding the feasible negative direction, λ_{min} which equals -0.290. This now creates a line segment, passing through the centre of the simplex defining all the feasible alternative weights that can be added to $\mathbf{a}$, $\{\lambda \tilde{\mathbf{a}} : \lambda_{min} \leq \lambda \leq \lambda_{max}\}$, such that $\mathbf{a}^T \mathbf{T} = (\mathbf{a} + \lambda \tilde{\mathbf{a}})^T \mathbf{T}$. This line segment is displayed in Figure 3.1, showing that the set of feasible alternative weights lies along the intersection of the null space of $[\mathbf{T}, \mathbf{1}]$ and the 3-simplex.

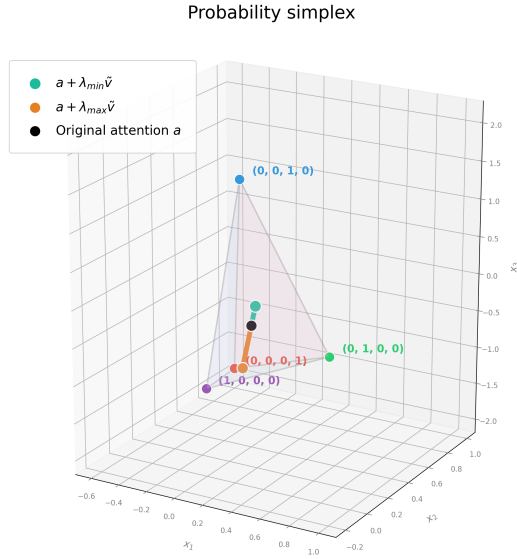


FIGURE 3.1. The set of alternative weights for the uniform distribution in the 3-simplex with matrix $\mathbf{T} \in \mathbb{R}^{4 \times 2}$.

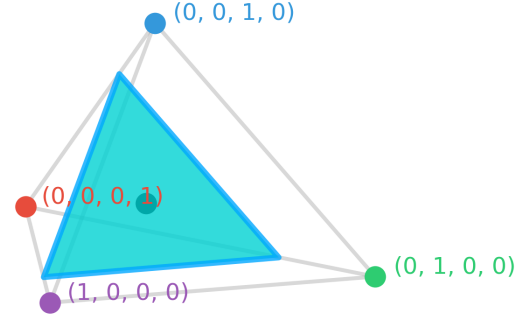


FIGURE 3.2. The set of alternative weights for the uniform distribution with matrix $\mathbf{T}' \in \mathbb{R}^{4 \times 1}$.

207 Additionally, one could consider a matrix $\mathbf{T}' = \begin{bmatrix} 1 & 1 & 1 & -3 \end{bmatrix}^T \in \mathbb{R}^{4 \times 1}$, then the rank of $[\mathbf{T}', \mathbf{1}]$ is 2,
 208 hence we expect the left null space of $[\mathbf{T}', \mathbf{1}]$ to be of dimension $4 - 2 = 2$. We go through the same
 209 process as for the one dimensional case to calculate the set of alternative weights. We find that this set is
 210 indeed a 2-dimensional subspace of the 3-simplex, as seen in Figure 3.2. Again, this is exactly the plane
 211 described by $\text{LN}([\mathbf{T}', \mathbf{1}])$ intersecting the 3-simplex. These two examples show that the set of alternative
 212 weights is the intersection of the null space and simplex.

213 **3.3. Effective Attention.** The above discussion has shown that when the sequence length is larger
 214 than the attention head dimension, we lose the uniqueness of the attention weights. In fact, once we have
 215 non-identifiability, we get an infinite set of alternative weights. Since for any vector in the left null space,
 216 a scalar multiple of it is also in the left null space. The work by [4] proposes a solution to this called
 217 *effective attention*. Suppose $\mathbf{T}$ has a non-trivial left null space, then one can decompose an attention
 218 weight matrix into components parallel and orthogonal to the null space,

$$219 \quad \mathbf{AT} = (\mathbf{A}_p + \mathbf{A}_o)\mathbf{T} = \mathbf{A}_p\mathbf{T},$$

220 since $\mathbf{A}_p \in \text{LN}(\mathbf{T})$ and so $\mathbf{A}_p\mathbf{T} = \mathbf{0}$. Thus, the component of the attention weights in $\text{LN}(\mathbf{T})$ do not
 221 contribute to the output and are deemed irrelevant. Therefore, the idea is to consider only the component
 222 orthogonal to the null space,

$$223 \quad (3.4) \quad \mathbf{A}^{eff} = \mathbf{A} - \text{Proj}_{\text{LN}(\mathbf{T})}\mathbf{A}.$$

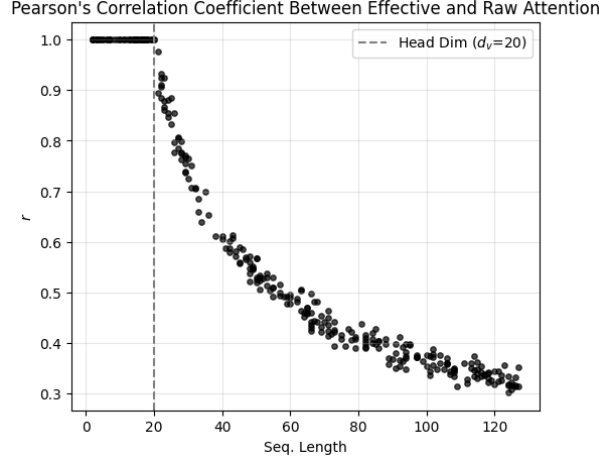


FIGURE 3.3. Pearson's correlation coefficient between effective and raw attention. The head dimension of the model is marked at a sequence length of 20.

Numerically, effective attention is solved by calculating the SVD of $\mathbf{T}$, $\mathbf{T} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, and finding the number of non zero singular values in the diagonal of $\mathbf{\Sigma}$. This gives $R = \text{rank}(\mathbf{T})$. The last $n - R$ columns, where n is the token sequence length, of $\mathbf{U}$ creates the orthogonal complement of the column space of $\mathbf{T}$, $\mathbf{U}_{\text{null}}$. The orthogonal projection matrix is then $\mathbf{U}_{\text{null}}(\mathbf{U}_{\text{null}})^T$, thus $\text{Proj}_{\text{LN}(\mathbf{T})}\mathbf{A}$ just becomes $\mathbf{U}_{\text{null}}(\mathbf{U}_{\text{null}})^T\mathbf{A}$. Hence, effective attention is given by $\mathbf{A}^{\text{eff}} = \mathbf{A} - \mathbf{U}_{\text{null}}(\mathbf{U}_{\text{null}})^T\mathbf{A}$. The difference between the raw and effective attention is quantified using the Pearson's correlation coefficient [1],

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2}}.$$

It measures the strength of the linear correlation between the two attention representations. To demonstrate this, random $\mathbf{T}$ and $\mathbf{A}$ matrices are produced, with $\mathbf{A}$ being an output of softmax. Random sequence lengths are chosen from 2 to 128 with a head size of 20. In this setting a single head of attention is considered so $d = d_v$. First, 300 samples are generated, then effective attention is calculated, and finally r is calculated. The results are plotted in Figure 3.3 showing how effective attention deviates away from the raw attention once the sequence length exceeds 20. This is the dimension of the head size for the model. When this is exceeded, the $\text{LN}(\mathbf{T})$ becomes non empty since $n > d_v$, thus causing effective attention to deviate from the raw attention. This happens as more of the raw attention becomes irrelevant creating less correlation between the effective attention and raw attention. When the sequence length is less than d_v , the dimension of the left null space is zero, meaning effective and raw attention are exactly the same, as displayed by the Pearson's coefficient of $r = 1$.

One may notice that we are projecting $\mathbf{A}$ onto the left null space of $\mathbf{T}$ and not $[\mathbf{T}, \mathbf{1}]$ and so it is not guaranteed that $\mathbf{A}^{\text{eff}}$ exists in the probability simplex. The solution is not as simple as projecting $\mathbf{A}$ onto

244 $[\mathbf{T}, \mathbf{1}]$, as the component of $\mathbf{A}$ that is irrelevant is determined by $\text{LN}(\mathbf{T})$, not $\text{LN}([\mathbf{T}, \mathbf{1}])$. Since $\text{LN}([\mathbf{T}, \mathbf{1}])$
 245 is a subspace of $\text{LN}(\mathbf{T})$, i.e. $\text{LN}([\mathbf{T}, \mathbf{1}]) \subseteq \text{LN}(\mathbf{T})$ then projecting onto $\text{LN}([\mathbf{T}, \mathbf{1}])$ would still leave some
 246 irrelevant attention in $\mathbf{A}^{eff}$. The investigation of how one restricts $\mathbf{A}^{eff}$ to the probability simplex is left
 247 for future study.

248 **4. Low-rank Bottleneck in Attention Heads.** We now turn our attention to the other ‘side’ of
 249 our distribution matrix, $\mathbf{A}$, and what happens in relation to the softmax function introduced in Section
 250 2.2. More specifically, another problem faced by attention heads is when our head size, d_v , is less than the
 251 input sequence length, n , namely the *low-rank bottleneck* (also known as the *low-rank softmax bottleneck*).
 252 Following this introduction, in this section we will present some novel findings on the low-rank bottleneck
 253 and show some details on how to avoid it. In Section 5 we will present our results on the empirical
 254 presence of this bottleneck in a smaller model.

255 **THEOREM 4.1** (Representation Theorem [3]). *If $d_q = d_v = d \geq n$, then, given any positive stochastic*
 256 *matrix $\mathbf{A}$, there always exists $d \times d$ projection matrices $\mathbf{W}_K, \mathbf{W}_Q$ such that*

$$257 \quad (4.1) \quad \text{softmax} \left(\frac{(\mathbf{W}_K \mathbf{E})^\top \mathbf{W}_Q \mathbf{E}}{\sqrt{d_v}} \right) = \mathbf{A}.$$

258 *If $d_q = d_v = d < n$, there exist $\mathbf{E}$ and $\mathbf{A}$ such that Equation (4.1) does not hold for all $\mathbf{W}_Q, \mathbf{W}_K$.*

259 Note that while this is for a single head, size d , the same holds for multiple heads, each with embedding
 260 size d_v , where we write d to be the size of the embedding in the head. This means that when we project
 261 our sequence down onto each head, if the head size is too small, we cannot capture all contexts, i.e all
 262 distribution matrices $\mathbf{A}$. We refer to this as not being fully *expressive*. An intuitive, albeit reductive,
 263 explanation is that the matrix inside the softmax function is of maximum $\text{rank}(d_v) < \text{rank}(n)$, so may
 264 have difficulty representing a full rank $\mathbf{A}$. However, due to softmax being a non-linear transformation,
 265 this is not the full picture as we will see in Section 4.1.

266 **4.1. Determining When a Distribution Can Be Produced.** To further our understanding of
 267 the bottleneck we derived a method for determining if a matrix $\mathbf{A}$ can be produced by softmax, given
 268 some embedding matrix $\mathbf{E}$. For this, given that $\mathbf{W}_K$ and $\mathbf{W}_q$ are learnt, we assume freedom in choosing
 269 them. This section involves the derivation of the condition we found. Formally, the question is, “Given
 270 some embedding matrix $\mathbf{E}$ and some distribution $\mathbf{A}$, and freedom in the key and query matrices, when
 271 can we reach $\mathbf{A}$ from $\mathbf{E}$?”.

272 Firstly, we have $\mathbf{E} \in \mathbb{R}^{d \times n}$ (where $d < n$) which is our embedding matrix, $\mathbf{W}_K \in \mathbb{R}^{d_v \times d}$ and $\mathbf{W}_Q \in \mathbb{R}^{d_q \times d}$,
 273 our key and query weight matrices, with $d = d_v = d_q$ as d is the size of the embedding in the head. As

274 seen before, we define our distribution matrix, $\mathbf{A} \in \mathbb{R}^{n \times n}$, as

$$275 \quad \mathbf{A} = \text{softmax} \left(\frac{\mathbf{E}^T \mathbf{W}_{\mathbf{K}}^T \mathbf{W}_{\mathbf{Q}} \mathbf{E}}{\sqrt{d_v}} \right).$$

276 Denote $\mathbf{M} = \frac{\mathbf{E}^T \mathbf{W}_{\mathbf{K}}^T \mathbf{W}_{\mathbf{Q}} \mathbf{E}}{\sqrt{d_v}}$, and write $\mathbf{M} = [M_1, \dots, M_n]$ and $\mathbf{A} = [A_1, \dots, A_n]$ with $M_i, A_i \in \mathbb{R}^n$. As
 277 softmax is applied column-wise, we have $A_i = \text{softmax}(M_i)$, so

$$278 \quad (A_i)_j = \frac{\exp((M_i)_j)}{\sum_{k=1}^n \exp((M_i)_k)}$$

$$279 \quad \Rightarrow \ln((A_i)_j) = (M_i)_j - C,$$

280 where $C = \sum_{k=1}^n \exp((M_i)_k)$ so is constant along every entry of a column. Putting this in full matrix
 281 terms, attempting to invert softmax we get

$$282 \quad (4.2) \quad \ln \mathbf{A} = \frac{\mathbf{E}^T \mathbf{W}_{\mathbf{K}}^T \mathbf{W}_{\mathbf{Q}} \mathbf{E}}{\sqrt{d_v}} - \mathbf{1} \mathbf{c}^T,$$

283 Where $\mathbf{c} = [c_1, \dots, c_n]^T \in \mathbb{R}^n$ is a vector of constants. We aim to solve Equation (4.2) for the existence
 284 of a $\mathbf{c}$; if such a $\mathbf{c}$ exists, we can theoretically produce $\mathbf{A}$ from $\mathbf{E}$. The following work reduces Equation
 285 (4.2) to a simpler condition.

286 Consider the null space of $\mathbf{E}$, $\mathcal{N}(\mathbf{E}) = \{y \mid \mathbf{E}y = 0\}$. As $d < n$, $\mathcal{N}(\mathbf{E})$ can be represented using $n - d$
 287 basis vectors. Denote these basis vectors $\{\mathbf{b}_1, \dots, \mathbf{b}_{n-d}\}$, $\mathbf{b}_i \in \mathbb{R}^n$. Choose some basis for $\mathcal{N}(\mathbf{E})$ and

288 let $\mathbf{N} \in \mathbb{R}^{(n-d) \times n}$ be the matrix with rows being the basis of $\mathcal{N}(\mathbf{E})$, i.e $\mathbf{N} = \begin{bmatrix} \mathbf{b}_1^T \\ \vdots \\ \mathbf{b}_{(n-d)}^T \end{bmatrix}$. Then we have

$$289 \quad \mathbf{N} \mathbf{E}^T = \mathbf{0} \text{ and consequently } \frac{\mathbf{N} \mathbf{E}^T \mathbf{W}_{\mathbf{K}}^T \mathbf{W}_{\mathbf{Q}} \mathbf{E}}{\sqrt{d_v}} = \mathbf{0}.$$

290 We can use this null space trick to our advantage. Consider Equation (4.2), left multiplying by $\mathbf{N}$ gives

$$291 \quad (4.3) \quad \mathbf{N} \ln(\mathbf{A}) = \frac{\mathbf{N} \mathbf{E}^T \mathbf{W}_{\mathbf{K}}^T \mathbf{W}_{\mathbf{Q}} \mathbf{E}}{\sqrt{d_v}} - \mathbf{N} \mathbf{1} \mathbf{c}^T$$

$$292 \quad (4.4) \quad = -\mathbf{N} \mathbf{1} \mathbf{c}^T.$$

293 Again writing $\mathbf{A} = [A_1, \dots, A_n]$, with each $A_i \in \mathbb{R}^n$, Equation (4.3) reduces down to solving

$$294 \quad (4.5) \quad \begin{bmatrix} \mathbf{b}_1^T \ln(A_1) & \cdots & \mathbf{b}_1^T \ln(A_n) \\ \vdots & & \vdots \\ \mathbf{b}_{(n-d)}^T \ln(A_1) & \cdots & \mathbf{b}_{(n-d)}^T \ln(A_n) \end{bmatrix} = \begin{bmatrix} c_1 \sum_{i=1}^n (\mathbf{b}_1)_i & \cdots & c_n \sum_{i=1}^n (\mathbf{b}_1)_i \\ \vdots & & \vdots \\ c_1 \sum_{i=1}^n (\mathbf{b}_{(n-d)})_i & \cdots & c_n \sum_{i=1}^n (\mathbf{b}_{(n-d)})_i \end{bmatrix}.$$

295 for the existence of $\mathbf{c} = [c_1, \dots, c_n]^T$, as we know all $\mathbf{b}_i \in \mathbf{N}$ and all $\ln(A_i)$. Here, $\ln(A_i)$ is the element-wise
 296 natural logarithm of a vector. Furthermore, this method is basis invariant. If one wished to use another
 297 basis, which would form a different matrix $\mathbf{N}$, which we can denote $\mathbf{N}'$, we can describe $\mathbf{N}' = \mathbf{R} \mathbf{N}$, where

298 $\mathbf{R}$ is some invertible change of basis matrix. Then, we would end up with the same condition, specifically
 299 as,

$$\begin{aligned} 300 \quad & \mathbf{N}' \ln(\mathbf{A}) = -\mathbf{N}' \mathbf{1} c^T \\ 301 \quad & \iff \mathbf{R} \mathbf{N} \ln(\mathbf{A}) = -\mathbf{R} \mathbf{N} \mathbf{1} c^T \\ 302 \quad & \iff \mathbf{N} \ln(\mathbf{A}) = -\mathbf{N} \mathbf{1} c^T. \end{aligned}$$

303 So, if such a $\mathbf{c}$ exists for one choice of basis, it also exists for any other.

304 One can also solve for the possibility of the distribution, A_i , of one token (token i). As c_i exists only in
 305 the corresponding column of A_i , one can simply compare specific columns of Equation (4.5) and solve for
 306 c_i . If such a c_i exists, then the distribution for that token can be generated by $\mathbf{E}$.

307 **4.2. Avoiding the Low-Rank Bottleneck.** The paper ‘Low-Rank Bottleneck in Multi-head At-
 308 tention Models’ [3] proposes a fix for the transformer neural network to avoid experiencing the low-rank
 309 bottleneck. Using the typical heuristic of choosing $d_v = \frac{d}{h}$ (Multi-Head Attention), our attention heads
 310 experience the low-rank bottleneck whenever $h > \frac{d}{n}$ (which can be trivially shown). Consequently, in [3]
 311 the authors suggest decoupling head size from both the number of heads and overall embedding dimension,
 312 and choosing the head size to be larger than or equal to the input sequence length; thus directly avoiding
 313 the bottleneck. In theory, this ensures all attention heads are fully expressive. The authors of [3] refer
 314 to this as ‘Fixed Multi-Head Attention’. Comparing to Section 2.3, the attention producing layer now
 315 becomes

$$\begin{aligned} 316 \quad & \text{fixedhead}(\mathbf{E})_i = \text{softmax} \left(\frac{\mathbf{K}_i^T \mathbf{Q}_i}{\sqrt{d_v}} \right) \cdot \mathbf{V}_i, \\ 317 \quad & \text{FixedMultiHead}(\mathbf{E}) = \text{Concat}[\text{fixedhead}(\mathbf{E})_1, \dots, \text{fixedhead}(\mathbf{E})_h]. \end{aligned}$$

318 This gives us the final layer output as,

$$319 \quad \mathbf{Z} = \text{LayerNorm}(\mathbf{E} + \text{FixedMultiHead}(\mathbf{E}) \cdot \mathbf{H}_F).$$

320 Note that as $\text{fixedhead}(\mathbf{E})_i \in \mathbb{R}^{n \times d_v}$ then $\text{FixedMultiHead}(\mathbf{E}) \in \mathbb{R}^{n \times (d_v \cdot h)}$. Therefore, the matrix $\mathbf{H}_F$ is
 321 in $\mathbb{R}^{(d_v \cdot h) \times n}$ and additionally serves to reshape the attention output of FixedMultiHead to $\mathbb{R}^{n \times n}$ to be
 322 added to the original embedding.

323 The expressivity of this is superior to that of MultiHead Attention, as seen in the following theorem.
 324 We firstly write $f_{\mathbf{H}_M}^{d,h}(\mathbf{E}) = \text{MultiHead}(\mathbf{E}) \cdot \mathbf{H}_M$ and $g_{\mathbf{H}_F}^{d,h,d_v}(\mathbf{E}) = \text{FixedMultiHead}(\mathbf{E}) \cdot \mathbf{H}_F$ and define
 325 $\mathcal{F}^{d,h} = f_{\mathbf{H}_M}^{d,h}(\cdot)$ and $\mathcal{G}^{d,h,d_v} = g_{\mathbf{H}_F}^{d,h,d_v}(\cdot)$ to be the family of functions representing the possible outputs of
 326 our attention layer.

327 **THEOREM 4.2 ([3]).** *Let $n \geq 2$, $d \geq d_v$ and $h \geq \frac{d}{d_v}$. Consider a FixedMultiHead attention layer,*

328 $g_{\mathbf{H}_F}^{d,h,d_v}(\cdot)$, with parameters which satisfy the following:

329
$$\mathbf{H}_F \times \begin{bmatrix} \mathbf{W}_1^v \\ \vdots \\ \mathbf{W}_h^v \end{bmatrix} \text{ is full rank, and } (\mathbf{W}_i^k)^T (\mathbf{W}_i^q) = \mathbf{U}, \text{ a rank } d_v \text{ matrix, for all } i \in \{1, \dots, h\}.$$

330 Then, for any $f_{\mathbf{H}_m}^{d,h} \in \mathcal{F}^{d,h}$, there exists $\mathbf{E} \in \mathbb{R}^{d \times n}$ such that $f_{\mathbf{H}_m}^{d,h}(\mathbf{E}) \neq g_{\mathbf{H}_F}^{d,h,d_v}(\mathbf{E})$.

331 We will not go into detail on Theorem 4.2; however, what this means is that the expressive power of Fixed
332 Multi-Head is strictly greater than that of Multi-Head [3].

333 **4.3. Implementation of Fixed Multi-head Attention.** To explore the low-rank bottleneck and
334 perform our own experiments, we adapted the NanoGPT model from [9] to implement Fixed Multi-Head
335 Attention (code available at [7]). This mainly involved rewiring the internal dimensions, and including
336 projections to account for the decoupling of head size to the number of heads and overall embedding
337 dimension. A visualisation of a Fixed Multi-Head layer in our model is shown in Figure 4.1. Section 5
provides details on our implementation and experiments.

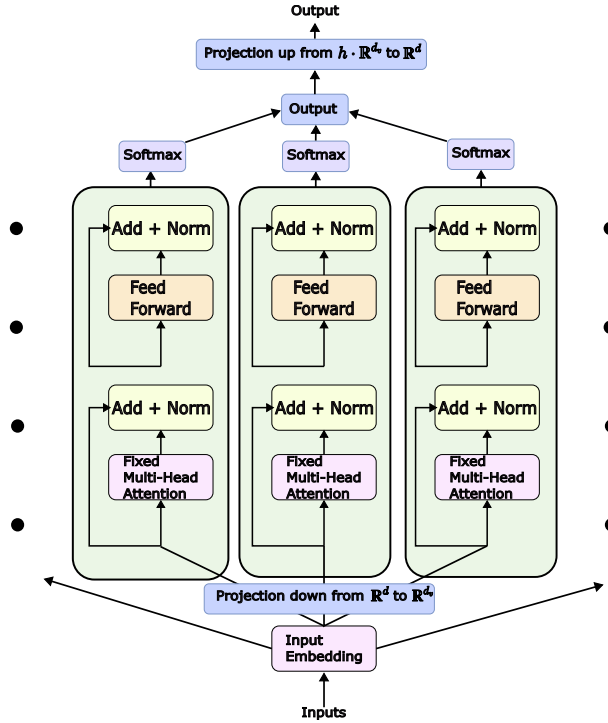


FIGURE 4.1. Simplified visualisation of Fixed Multi-Head Attention layer

338

339 **5. Experiments.** In our experiments, we used NanoGPT [9], a small transformer-based model,
340 trained on character-level embeddings in order to run training and inference on laptop CPUs. Each

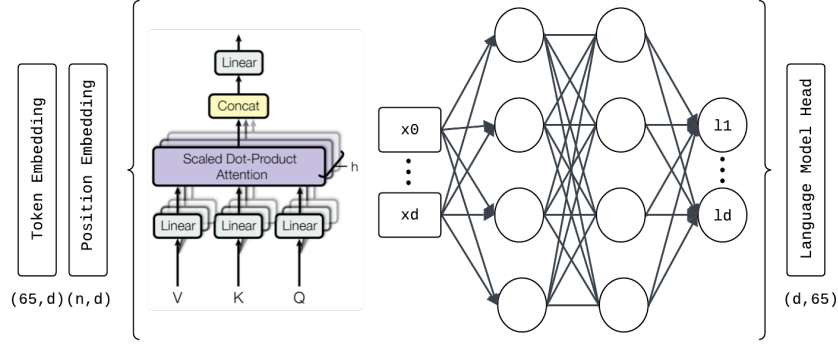


FIGURE 5.1. *GPT architecture using character-level embeddings.*

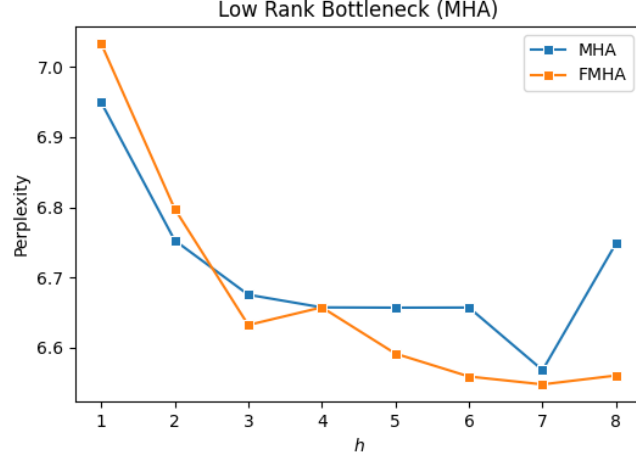


FIGURE 5.2. *Relative Performance of NanoGPT models for fixed parameters $d = 128$, $n = 32$.*

transformer block in the GPT architecture is comprised of a multi head attention (MHA) layer followed by a multi-layer perceptron (MLP). After two embedding layers (see Section 2.1 for details), transformer layers are connected in series (see Figure 5.1). Finally the language model head maps from the embedding dimension d , back into the vocabulary set C defined in Equation (2.1), producing a normalised probability distribution for the next token. Note that after each of these layers, a dropout and layer-normalisation is applied.

5.1. Low-Rank Bottleneck Numerics. We seek to demonstrate the benefits of fixed multi-head attention in terms of attention head expressivity. Consider two sets of GPT models, each with $\ell = 3$ transformer blocks, a sequence length of $n = 32$ tokens, embedding dimension $d = 128$, and a inter-layer dropout of 20%. In the first set of models (denoted MHA), each model's transformer block consists of a multi-head attention following the original heuristic of head size $d_v = \lfloor d/h \rfloor$. The second set of models (denoted FMHA) fixes the head size $d_v = 40 > n$.

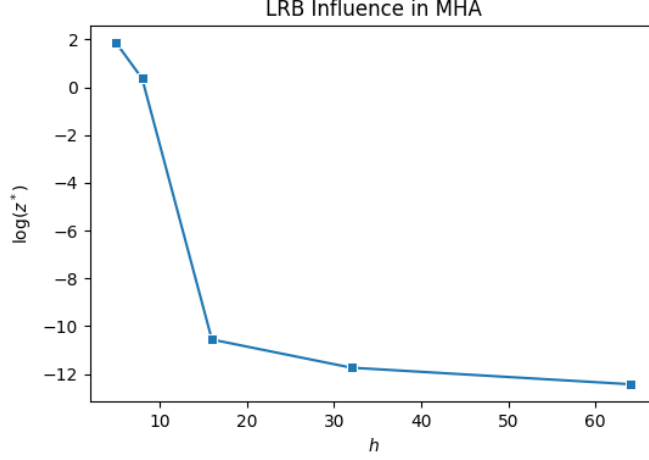


FIGURE 5.3. Removability of LRB for GPT models with fixed parameters $d = 128$, $n = 32$.

We then vary the number of heads in each model, and estimate the average perplexity P as follows. Consider the vocabulary list C defined in (2.1). Let V be the length of the vocabulary list C , i.e. the vocabulary size, here $V = 65$. Let $\mathcal{G} : \mathbb{R}^d \rightarrow [0, 1]^V$ be a function representing a forward pass through the GPT model from token embeddings. The image of $\mathcal{G}$ is a normalised probability distribution for the next token. Let $\mathcal{H} : \{1, \dots, V\}^n \rightarrow \{0, 1\}^V$ be a function that one-hot encodes an index into the vocabulary list C . Consider a one-hot encoding $\mathbf{y}_n \in \{0, 1\}^V$ where $y_{n,c} = \mathbb{I}(y_n = c)$ indicates whether index y_n corresponds to $c \in C$. Then, cross-entropy loss is given by,

$$\mathcal{L}(\mathbf{y}, \mathbf{p}) = \sum_{c=1}^{|C|} \mathbf{y}_c \ln(\mathbf{p}_c).$$

Averaging over $M = 350$ random samples from the evaluation dataset, we obtain the mean perplexity,

$$P = \frac{1}{M} \sum_{i=1}^M \exp \left\{ \mathcal{L}(\mathcal{H}(Y_i), \mathcal{G}(X_i)) \right\},$$

where $X_i \in \mathbb{R}^{d \times n}$ is token embeddings and $Y_i \in \mathbb{N}^n$ is the index of the label token in the vocabulary list C . Although MHA and FMHA models perform similarly for $h \leq d/n$, models following the fixed multi-head heuristic have a strictly lower perplexity for $h > d/n = 4$ (see Figure 5.2). When the number of heads $h > d/n$, the attention unit inside each head projects onto a dimension smaller than n , creating a low-rank bottleneck (LRB). Under the original multi head heuristic, increasing the number of heads simultaneously decreases the head size, and when $d_v < n$ performance degrades. The fixed head implementation addresses this issue by fixing $d_v = 40 > n$, which leads to increased model accuracy (i.e. lower perplexity) for when $h > \lfloor d/n \rfloor$.

5.2. Removing the Low-Rank Bottleneck in MHA. In this section, we attempt to unify both the findings on identifiability and the low-rank bottleneck in our paper. We attempted to see if identi-

fiability could help mitigate the problems caused by the low-rank bottleneck. We set out to see if, for distributions $\mathbf{A}$ that we cannot reach due to the low-rank bottleneck, there existed alternative distributions, due to non-identifiability, which we could reach that also resulted in the same attention head output. The following outlines our method.

Consider a GPT model following the original MHA heuristic with input sequence length n , embedding dimension d and h heads, each of size $d_v = \lfloor d/h \rfloor < n$. Since this model currently experiences a low-rank bottleneck, the distributions $\mathbf{A}_j : j \in \{1, \dots, n\}$ are non-identifiable. Thus, we seek an attention matrix $\mathbf{A}' = \mathbf{A} + \tilde{\mathbf{A}}$ that eliminates the low-rank bottleneck, while producing the same attention-head output.

First, we revisit Section 4.1 to formulate when a single probability vector (distribution) can be reached. Let augmented matrix $T_{\text{aug}} = \begin{bmatrix} \mathbf{T} & \mathbf{1} \end{bmatrix} \in \mathbb{R}^{n \times (d_v + 1)}$ where $\mathbf{T} = W^V E H_M \in \mathbb{R}^{n \times d_v}$, and consider two matrices $\mathbf{N}$ and $\mathbf{N}_{\text{aug}}$ given by,

$$\mathbf{N}_{\text{aug}} = \begin{bmatrix} \mathbf{y}_1, \dots, \mathbf{y}_k \end{bmatrix} \quad \text{where, } k = n - d_v - 1 \quad \text{and} \quad \mathbf{N} = \begin{bmatrix} \mathbf{v}_1^\top \\ \vdots \\ \mathbf{v}_\ell^\top \end{bmatrix} \quad \text{where } \ell = d - n.$$

Where $\mathbf{v}_i$ is a basis vector of $\text{LN}(\mathbf{E}^\top) \subset \mathbb{R}^n$ for $i \in \{1, \dots, \ell\}$ and $\mathbf{y}_i$ is a basis vector of $\text{LN}(\mathbf{T}_{\text{aug}}) \subset \mathbb{R}^n$ for $i \in \{1, \dots, k\}$. Given embedding matrix $\mathbf{E} \in \mathbb{R}^{d \times n}$ and attention matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, if $\exists \mathbf{c} \in \mathbb{R}^n$ such that $\mathbf{N} \ln(\mathbf{A}) = -\mathbf{N} \mathbf{1} \mathbf{c}^\top$, then there exists key and query weight matrices $\mathbf{W}_k$ and $\mathbf{W}_q$ such that,

$$\text{softmax} \left(\frac{(\mathbf{W}_k \mathbf{E})^\top \mathbf{W}_q \mathbf{E}}{\sqrt{d_v}} \right) = \mathbf{A}.$$

Next, we revisit Section 3 to determine what alternative probability vectors lead to the same column of the attention-head output. Let $\mathbf{a} := \mathbf{A}_j$ for some $j \in \{1, \dots, n\}$ be a column vector of the attention matrix $\mathbf{A}$, then $\mathbf{a}$ is a probability vector resulting from a softmax transformation, so each of its components is strictly positive. Consider a perturbation $\tilde{\mathbf{a}} = \mathbf{N}_{\text{aug}} \boldsymbol{\lambda} : \boldsymbol{\lambda} \in \mathbb{R}^k$ such that $\mathbf{a} + \tilde{\mathbf{a}} \geq \mathbf{0}$. Then $\tilde{\mathbf{a}} \in \text{LN}(\mathbf{T}_{\text{aug}})$ so conditions 3.2(a) and 3.2(b) are satisfied, and since $\mathbf{a} + \tilde{\mathbf{a}} \geq \mathbf{0}$, condition 3.2(c) is also satisfied. Hence, we can construct a polytope $\mathcal{P}_j := \{\mathbf{a} \in \mathbb{R}^n \mid \mathbf{A}_j + \mathbf{N}_{\text{aug}} \boldsymbol{\lambda} \geq \mathbf{0}, \boldsymbol{\lambda} \in \mathbb{R}^k\} \subset \mathbb{R}^n$ that represents the set of probability vectors which produce the same attention-head output as $\mathbf{a}$ i.e. $\mathbf{a} \mathbf{T} = \mathbf{a}' \mathbf{T} \forall \mathbf{a}' \in \mathcal{P}_j$. Let the probability vector $\mathbf{a}' \in \mathcal{P}_j$, then in order for that distribution to be reached, we must have that,

$$\mathbf{v}_i^\top \ln(\mathbf{a}) = \mathbf{v}_i^\top \mathbf{1} c_j \forall i \in \{1, \dots, \ell\} \iff \frac{\mathbf{v}_i^\top \ln(\mathbf{a})}{\mathbf{v}_i^\top \mathbf{1}} = \phi_i(\mathbf{a}) = c_j \forall i \in \{1, \dots, \ell\},$$

where $\phi_i(\mathbf{x}) := \mathbf{v}_i^\top \ln(\mathbf{x}) / \mathbf{v}_i^\top \mathbf{1}$ for $i \in \{1, \dots, \ell\}$.

DEFINITION 5.1. A low-rank bottleneck (LRB) is said to be ‘associated’ with column j of attention matrix $\mathbf{A}$ if there does not exist a probability vector $\mathbf{a}' \in \mathcal{P}_j$ such that $\max_{i \in \{1, \dots, \ell\}} |\phi_i(\mathbf{a}') - \phi_i(\mathbf{a})| < \eta$ for some tolerance level $0 < \eta \ll 1$.

We now seek a method to eliminate the LRB associated with a single column of $\mathbf{A}$. Let $f : \mathbb{R}^k \rightarrow \mathbb{R}$ given by $f(\boldsymbol{\lambda}) = \max_{i \in \{1, \dots, \ell\}} |\phi_i(\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda}) - \phi_1(\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda})|$ then $f(\boldsymbol{\lambda})$ measures how close we are to eliminating the low-rank bottleneck associated with column $\mathbf{A}_j := \mathbf{a}$. Hence, we seek a linear combination of basis vectors of $\text{LN}(T_{\text{aug}}^\top)$, given by $\boldsymbol{\lambda} \in \mathbb{R}^k$, which minimises objective function f , from which we construct a probability vector $\mathbf{a}' = \mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda}$ that leads to the same attention-head output. We formulate this in the following non-linear optimisation problem,

$$(P) \quad \min_{\boldsymbol{\lambda} \in \mathbb{R}^k} \max_{i \in \{1, \dots, \ell\}} |\phi_i(\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda}) - \phi_1(\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda})| \quad \text{s.t.} \quad \mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda} \geq \mathbf{0}.$$

Since the objective is non-linear, we proceed with an interior point method [5] followed by least-squares sequential quadratic programming (SLSQP) with $K = 50$ restarts, implemented via `scipy.optimize.minimize(method='SLSQP')`. Briefly, in SLSQP, the quadratic sub-problem approximates the non-linear objective, and the search direction is found by solving a least-squares problem [11]. First, we obtain a starting interior point $\boldsymbol{\lambda} \in \mathbb{R}^k$ by solving the following sub-problem,

$$(LP) \quad \min_{(\boldsymbol{\lambda}, s)} s \quad \text{s.t.} \quad \mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda} \geq s \cdot \mathbf{1}, \quad \boldsymbol{\lambda} \in \mathbb{R}^k, \quad s \geq 0.$$

Using the optimal solution $\boldsymbol{\lambda}^* \in \mathbb{R}^k$, the initial starting point for the next sub-problem is given by $\boldsymbol{\lambda}_0 = \boldsymbol{\lambda}^* + \epsilon$ where $\epsilon = (\epsilon_0, \dots, \epsilon_k) : \epsilon_i \sim \mathcal{N}(0, 0.01) \forall i \in 1, \dots, k$ such that $\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda}_0 \geq \mathbf{0}$. Thus,

$$(SLSQP) \quad \min_{\boldsymbol{\lambda} \in \mathbb{R}^k} \max_{i \in \{1, \dots, \ell\}} |\phi_i(\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda}) - \phi_1(\mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda})| \quad \text{s.t.} \quad \mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda} \geq \bar{\eta} \cdot \mathbf{1}$$

where $0 < \bar{\eta} \ll 1$ is another tolerance level. After training a GPT model with parameters ($d = 128$, $n = 32$, $h = 64$, $d_v = 2$), we sought to remove the LRB associated with column 1. When solving (P) with $\bar{\eta} = 1 \times 10^{-8}$, we obtained an optimal value of $z_1^* := f(\boldsymbol{\lambda}^*) = 4 \times 10^{-6}$. Note that by construction, $\mathbf{a}' = \mathbf{a} + \mathbf{N}_{\text{aug}}\boldsymbol{\lambda}^* \in \mathbb{R}^n$ is a probability vector that leads to the same attention-head output as $\mathbf{a} = \mathbf{A}_1$. So, we have eliminated the low-rank bottleneck associated with the first column of $\mathbf{A}$ at tolerance level $\eta = 10^{-5}$.

Next, we seek to remove the low-rank bottleneck associated with all columns of $\mathbf{A}$ via this method, which entails solving problem (P) n times over. Alas, after attempting to remove the LRB for several attention matrices $\mathbf{A}$ and embedding matrices $\mathbf{E}$, we arrived at the following conclusion. Given fixed matrices $\mathbf{A}$ and $\mathbf{E}$, although we may effectively eliminate the low-rank bottleneck associated with a subset S of columns (at tolerance $\eta = 10^{-4}$), we could not simultaneously eliminate the LRB associated with the remaining columns. In other words, in each trial, we obtained $z_j^* < \eta \forall j \in S$ and $z_i^* \geq \eta$ for all $i \in \{1, \dots, n\} \setminus S$. Finally, we trained 6 GPT models following the original MHA heuristic, each with $d = 128$ and $n = 32$ but varying $h \in \{5, 8, 16, 32, 64\}$, and solved (P) for a sample of batches from the evaluation set, recording the smallest optimal value z_1^* across all batches. As $|n - d_v|$ increases, the influence of the low-rank bottleneck worsens, since each attention head projects onto a smaller subspace, however, it also becomes easier to

eliminate the low-rank bottleneck associated with the first column (see Figure 5.3). Nevertheless, it still remains difficult to simultaneously remove the low-rank bottleneck induced by *all* the columns of the $\mathbf{A}$. Hence, we conclude that the LRB is an irrecoverable architectural flaw in GPT models following the MHA heuristic when head size $d_v = d/h < n$.

6. Conclusion. Identifiability of distributions and the low-rank bottleneck are two key problems which arise in transformer neural networks. This research has provided insight into how and why these problems occur, along with a novel method to determine if a distribution matrix is affected by the low-rank bottleneck. The theory of identifiability and the low-rank bottleneck is successfully implemented into a NanoGPT model trained on Shakespearian text, and numerical experiments carried out. Furthermore, an attempt to tie both areas together is made within this model. Conclusive evidence to support the theory around these issues explored in [4, 3] is also shown in this model.

On the identifiability side, we established that attention weights are non-identifiable when the input sequence length exceeds the attention head dimension, $n > d_v$. If the left null space of $\mathbf{T}$ is non empty, an infinite number of feasible alternative attention weights exists, each producing an identical output. This was visualised on the probability simplex for given examples. Effective attention is introduced that ignores the weights that exists in $\text{LN}(\mathbf{T})$. The difference between raw and effective attention was quantified using Pearson’s correlation coefficient where the exact point where they differ aligns with d_v . This result coincides with our finding in Theorem 3.2.

On the expressiveness side, the low-rank bottleneck creates a different problem. We introduced the low-rank bottleneck and, rather than asking whether attention weights are unique, asked whether a desired attention distribution is producible or not, given an embedding $\mathbf{E}$. We formalised a novel condition for determining reachability by exploiting the null space of the embedding matrix, reducing the problem to verifying the existence of a vector of constants $\mathbf{c}$ satisfying $\mathbf{N} \ln(\mathbf{A}) = -\mathbf{N} \mathbf{1} \mathbf{c}^T$. We demonstrated through NanoGPT experiments that the Fixed Multi-Head Attention architecture, which decouples head size from the number of heads, avoids the bottleneck entirely and achieves lower perplexity than standard Multi-Head Attention when $h > d/n$ indicating a better performing model.

Finally, an attempt was made to join both problems by asking whether the non-identifiability of attention could be exploited to eliminate the low-rank bottleneck. We showed that for a single column of the attention matrix, an optimisation problem covering the entire null space of $[\mathbf{T}, \mathbf{1}]$ can find an equivalent probability vector that removes the bottleneck, achieving an optimal value of $z^* = 4 \times 10^{-6}$. However, this approach does not generalise for all n columns. We therefore conclude that the low-rank bottleneck is an irrecoverable architectural limitation of standard Multi-Head attention when $d_v = d/h < n$.

However, there are limitations to our study. To effectively train a GPT model, we require far more computing power than what was available. For instance, GPT2, a 124 million parameter model, takes 3.5

439 days to train on a cluster of 8 A100 40GB nodes using a BPE token embedding on the OpenWebText2
440 dataset [8] [12]. Since we were constrained to use laptop CPUs, we could only train very small GPT
441 models using character-level embeddings, which limited the output of these networks.
442 There are several areas where further work can take place. Effective attention can be explored further to
443 see the effects it has on our NanoGPT model and see if the idea can be extended across to Multi-Head
444 attention. Additionally, further work can be put in to see if effective attention can be constrained to the
445 probability simplex. The use of Radon partitions would also be beneficial to explore to see if alternative
446 weights can be explicitly stated and formulated.

References.

- [1] Norah D. Alshahrani. “Statistical reproducibility of correlation tests: Pearson, Spearman, and Kendall”. In: *AIMS Mathematics* 11.1 (2026), pp. 957–976. DOI: [10.3934/math.2026042](https://doi.org/10.3934/math.2026042). URL: <https://doi.org/10.3934/math.2026042>.
- [2] *ASCII Code – The Extended ASCII Table*. <https://www.ascii-code.com/>. Accessed: 2026-04-16.
- [3] Srinadh Bhojanapalli et al. “Low-Rank Bottleneck in Multi-head Attention Models”. In: *Proceedings of the 37th International Conference on Machine Learning*. Ed. by Hal Daumé III and Aarti Singh. Vol. 119. Proceedings of Machine Learning Research. PMLR, 13–18 Jul 2020, pp. 864–873. URL: <https://proceedings.mlr.press/v119/bhojanapalli20a.html>.
- [4] Gino Brunner et al. “On Identifiability in Transformers”. In: *International Conference on Learning Representations*. 2020. URL: <https://openreview.net/forum?id=BJg1f6EFDB>.
- [5] Richard H. Byrd, Mary E. Hribar, and Jorge Nocedal. “An Interior Point Algorithm for Large-Scale Nonlinear Programming”. In: *SIAM Journal on Optimization* 9.4 (1999), pp. 877–900. DOI: [10.1137/S1052623497325107](https://doi.org/10.1137/S1052623497325107).
- [6] Jacob Devlin et al. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL 2019)*. 2019, pp. 4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
- [7] Banarse F. et al. *nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs*. <https://github.com/ardenpalme/nanoGPT>. Accessed: 2026-04-16. 2026.
- [8] Leo Gao et al. “The Pile: An 800GB Dataset of Diverse Text for Language Modeling”. In: *arXiv preprint arXiv:2101.00027* (2020).
- [9] Andrej Karpathy. *nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs*. <https://github.com/karpathy/nanoGPT>. Accessed: 2026-04-16. 2022.
- [10] Andrej Karpathy. *tinysakespeare dataset*. <https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinysakespeare/input.txt>. Accessed: 17 April 2026. 2015.
- [11] Dieter Kraft. *A Software Package for Sequential Quadratic Programming*. Tech. rep. DFVLR-FB 88-28. Köln, Germany: DLR German Aerospace Center – Institute for Flight Mechanics, 1988.
- [12] Alec Radford et al. “Language Models are Unsupervised Multitask Learners”. In: *OpenAI Blog* 1.8 (2019). URL: <https://openai.com/blog/better-language-models>.
- [13] Ashish Vaswani et al. “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017.